

Web Scraping Example - IMDB Top 250 Movies

Adrianne Bradford

February 25, 2019

Installing XML and rvest

First, we need to load the package used in R for web scraping, rvest, and a package for parsing XML, called XML.

```
if (!requireNamespace("XML", quietly = TRUE)) {
  install.packages("XML")
}
if (!requireNamespace("rvest", quietly = TRUE)) {
  install.packages("rvest")
}

library('XML')
library('rvest')
```

The Website

We are going to scrape the page which is a listing of the top 250 movies on IMDB. We will obtain the names of the movies, the URL for more information about the movie, the year of the movie, the rating, runtime, genre, and the star rating. First, we set up a string variable, called "site" and save the URL of the website we will scrape to it. This will become an argument we pass to an R function to return the HTML results we will parse for the data we seek. In the next line we use the read_html function to perform this action.

```
site <- "https://www.imdb.com/search/title?groups=top_250&sort=user_rating"
sitehtml <- read_html(site)
sitehtml
```

```
## {xml_document}
## <html xmlns:og="http://ogp.me/ns#" xmlns:fb="http://www.facebook.com/2008/fbml">
## [1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset= ...
## [2] <body id="styleguide-v2" class="fixed">\n\n      <img height=" ...
```

This doesn't look like a lot of output, but all of the code for the site is contained in these two objects - one for the head part of the page, the other for the body.

Looking just at the body, we see more of the code, but again large sections of the page are contained in each of these items.

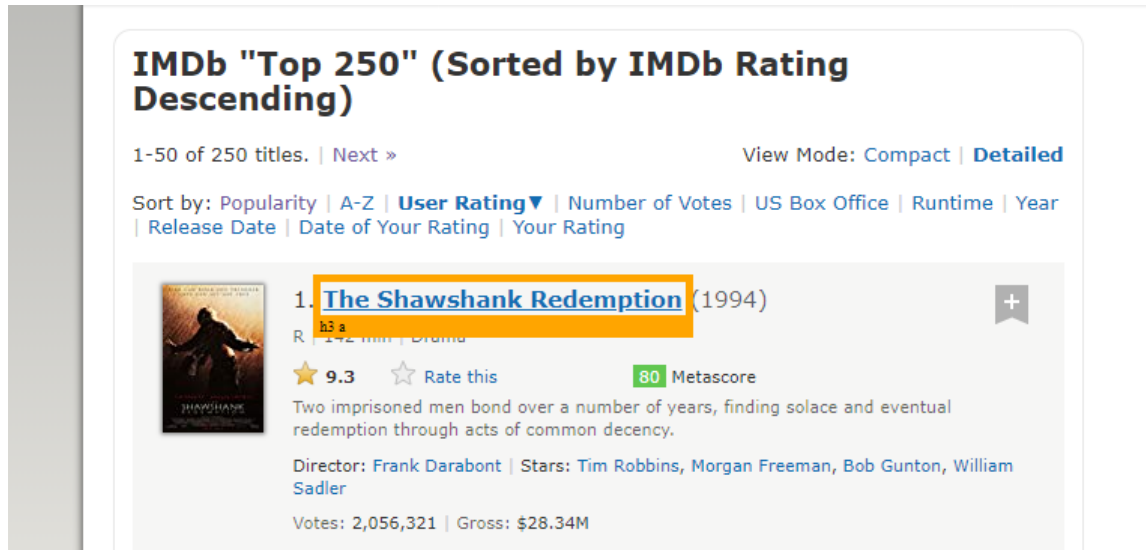
```
html_node(sitehtml, 'body')
```

```
## {xml_node}
## <body id="styleguide-v2" class="fixed">
## [1] <img height="1" width="1" style="display:none;visibility:hidden;" s ...
## [2] <script>\n    if (typeof uet == 'function') {\n      uet("bb");\n    ...
## [3] <script>\n    if ('csm' in window) {\n      csm.measure('csm_body_d ...
## [4] <div id="sis_pixel">\n      <!-- begin sis pixel slot --> ...
## [5] <div id="wrapper">\n      <div id="root" class="redesign">\n    ...
## [6] <script type="text/javascript" src="https://m.media-amazon.com/imag ...
## [7] <script type="text/javascript" src="https://m.media-amazon.com/imag ...
## [8] <script type="text/javascript" src="https://m.media-amazon.com/imag ...
## [9] <script type="text/javascript" src="https://m.media-amazon.com/imag ...
## [10] <script type="text/javascript" src="https://m.media-amazon.com/imag ...
## [11] <script type="text/javascript" src="https://m.media-amazon.com/imag ...
## [12] <script type="text/javascript">\n      function jQueryOnReady ...
## [13] <script>\n\tif (typeof doWithAds === "function") { \n\t  doWithAd ...
## [14] <div id="servvertime" time="174"></div>
## [15] <script>\n    if (typeof uet == 'function') {\n      uet("be");\n    ...
## [16] <div id="be" style="display:none;visibility:hidden;">\n<form name=" ...
## [17] <noscript>\n      <img height="1" width="1" style="display:none;visib ...
```

In order to find the pieces of the page we want, we need to dig into the source code and locate exactly where the pieces we need are and pick those out.

Planning the Scrape

Now we review our website in a web browser to determine where in the HTML code the pieces we want to scrape are located. This can be done by using the "Developer Tools" built into Google Chrome (similar tools available in Firefox as well) by right clicking on an item you want to scrape and clicking "Inspect." Another option is to install the SelectorGadget extension for Google Chrome (<https://selectorgadget.com/> (<https://selectorgadget.com/>)). When that extension is enabled you can click and highlight items of interest on the webpage and see the html nodes they are located in. For this first one we'll use the selectogGadget, as pictured below.



With SelectorGadget enabled, we mouse over the name of the first movie, and the orange square that shows up tells us that the name of the movie is in the `h3 a` tag. [h3 is header level 3, a is an anchor tag that usually precedes a link (href)]

First Row of the Table

So first, let's pull back the first row of content using the SelectorGadget to identify the tag for the movie names to see what we get. We'll use `html_node` to just get the first instance of the item on the page.

```
firstrow <- html_node(sitehtml, 'h3 a')
firstrow
```

```
## {xml_node}
## <a href="/title/tt0111161/?ref=adv_li_tt">
```

```
html_text(firstrow)
```

```
## [1] "The Shawshank Redemption"
```

This looks like the information we want, the name of the first movie on the list and the link to that movie's detailed page on IMDB. Note that the link doesn't include the full link of the movie's page, starting with `http://www.imdb.com` (`http://www.imdb.com`), this is because it is a relative link, relative to your current position in the website. If you were to link to that page from your own site, you would need to use an absolute link, with the full `http://www.imdb.com` (`http://www.imdb.com`) included.

Get all Rows

The function `html_node` pulled down one item, the first instance of what we were looking for, and it was the correct item. If we want to pull down all of the items, we use the function `html_nodes` (note the plural) to get all the `<h3 a>` objects from the website.

```
body_rows <- html_nodes(sitehtml, "h3 a")
## PRINT FIRST 4 ITEMS
body_rows[1:4]
```

```
## {xml_nodeset (4)}
## [1] <a href="/title/tt0111161/?ref=adv_li_tt">The Shawshank Redemption</a>
## [2] <a href="/title/tt0068646/?ref=adv_li_tt">The Godfather</a>
## [3] <a href="/title/tt0468569/?ref=adv_li_tt">The Dark Knight</a>
## [4] <a href="/title/tt0071562/?ref=adv_li_tt">The Godfather: Part II</a>
```

We know there were only 50 movies, but there are 55 items in our list of body rows. Looking at the tail of the list (the last 6 items) we confirm the last 5 are likely special links for ads/etc. at the bottom of the page.

```
length(body_rows)
```

```
## [1] 55
```

```
tail(body_rows)
```

```
## {xml_nodeset (6)}
## [1] <a href="/title/tt0078788/?ref=adv_li_tt">Apocalypse Now</a>
## [2] <a href="/offsite/?page-action=fol_fb&token=BCYu_NAoZynWZuDSgD3T ...
## [3] <a href="/offsite/?page-action=fol_inst&token=BCYg3Ihsz8pShADrjt ...
## [4] <a href="/offsite/?page-action=fol_twch&token=BCYtjU096Aim_zX29p ...
## [5] <a href="/offsite/?page-action=fol_tw&token=BCYozmGwVE6Fiq140yXb ...
## [6] <a href="/offsite/?page-action=fol_yt&token=BCYhv_iKk6Gqc0YXN0ko ...
```

Getting just the bits we want

We have our listing of the first 50 movie titles and links, but they're stuck together in a cell with a bunch of html "stuff" around them too.

We want to extract from our `body_rows` two lists with just the pieces we want to keep, one with the titles and one with the URLs.

We obtain the titles by using the `html_text` function on just our `body_rows` object (not the full webpage). This function pulls out all the text that is not part of the HTML tags - it pulls everything between the open tag `<a>` and the close tag `</a>`. It pulls back 55 items, 5 of which are our blank items that are at the end of the page and we determined are not movies, so we trim those off the end of our list of titles.

```
titles <- html_text(body_rows)
length(titles)
```

```
## [1] 55
```

```
titles <- titles[1:50]
titles
```

```
## [1] "The Shawshank Redemption"
## [2] "The Godfather"
## [3] "The Dark Knight"
## [4] "The Godfather: Part II"
## [5] "The Lord of the Rings: The Return of the King"
## [6] "Pulp Fiction"
## [7] "Schindler's List"
## [8] "The Good, the Bad and the Ugly"
## [9] "12 Angry Men"
## [10] "Inception"
## [11] "Fight Club"
## [12] "The Lord of the Rings: The Fellowship of the Ring"
## [13] "Forrest Gump"
## [14] "Star Wars: Episode V - The Empire Strikes Back"
## [15] "Spider-Man: Into the Spider-Verse"
## [16] "The Lord of the Rings: The Two Towers"
## [17] "The Matrix"
## [18] "Goodfellas"
## [19] "One Flew Over the Cuckoo's Nest"
## [20] "Seven Samurai"
## [21] "Andhadhun"
## [22] "Interstellar"
## [23] "City of God"
## [24] "Spirited Away"
## [25] "Saving Private Ryan"
## [26] "Life Is Beautiful"
## [27] "The Usual Suspects"
## [28] "Se7en"
## [29] "Léon: The Professional"
## [30] "The Silence of the Lambs"
## [31] "Star Wars"
## [32] "It's a Wonderful Life"
## [33] "Dangal"
## [34] "Avengers: Infinity War"
## [35] "Whiplash"
## [36] "The Intouchables"
## [37] "The Prestige"
## [38] "The Departed"
## [39] "The Pianist"
## [40] "Memento"
## [41] "Gladiator"
## [42] "The Green Mile"
## [43] "American History X"
## [44] "The Lion King"
## [45] "Terminator 2: Judgment Day"
## [46] "Cinema Paradiso"
## [47] "Grave of the Fireflies"
## [48] "Back to the Future"
## [49] "Raiders of the Lost Ark"
## [50] "Apocalypse Now"
```

We also need to capture the URLs separately. To do this we need to use the `html_attr` function to obtain the subset (attribute) of the `<a>` (anchor) object that contains the link `href=` and again only obtain the first 50 items.

```
urls <- (html_attr(body_rows, "href"))[1:50]
urls
```

```
## [1] "/title/tt0111161/?ref=adv_li_tt" "/title/tt0068646/?ref=adv_li_tt"
## [3] "/title/tt0468569/?ref=adv_li_tt" "/title/tt0071562/?ref=adv_li_tt"
## [5] "/title/tt0167260/?ref=adv_li_tt" "/title/tt0110912/?ref=adv_li_tt"
## [7] "/title/tt0108052/?ref=adv_li_tt" "/title/tt0060196/?ref=adv_li_tt"
## [9] "/title/tt0050083/?ref=adv_li_tt" "/title/tt1375666/?ref=adv_li_tt"
## [11] "/title/tt0137523/?ref=adv_li_tt" "/title/tt0120737/?ref=adv_li_tt"
## [13] "/title/tt0109830/?ref=adv_li_tt" "/title/tt0080684/?ref=adv_li_tt"
## [15] "/title/tt4633694/?ref=adv_li_tt" "/title/tt0167261/?ref=adv_li_tt"
## [17] "/title/tt0133093/?ref=adv_li_tt" "/title/tt0099685/?ref=adv_li_tt"
## [19] "/title/tt0073486/?ref=adv_li_tt" "/title/tt0047478/?ref=adv_li_tt"
## [21] "/title/tt8108198/?ref=adv_li_tt" "/title/tt0816692/?ref=adv_li_tt"
## [23] "/title/tt0317248/?ref=adv_li_tt" "/title/tt0245429/?ref=adv_li_tt"
## [25] "/title/tt0120815/?ref=adv_li_tt" "/title/tt0118799/?ref=adv_li_tt"
## [27] "/title/tt0114814/?ref=adv_li_tt" "/title/tt0114369/?ref=adv_li_tt"
## [29] "/title/tt0110413/?ref=adv_li_tt" "/title/tt0102926/?ref=adv_li_tt"
## [31] "/title/tt0076759/?ref=adv_li_tt" "/title/tt0038650/?ref=adv_li_tt"
## [33] "/title/tt5074352/?ref=adv_li_tt" "/title/tt4154756/?ref=adv_li_tt"
## [35] "/title/tt2582802/?ref=adv_li_tt" "/title/tt1675434/?ref=adv_li_tt"
## [37] "/title/tt0482571/?ref=adv_li_tt" "/title/tt0407887/?ref=adv_li_tt"
## [39] "/title/tt0253474/?ref=adv_li_tt" "/title/tt0209144/?ref=adv_li_tt"
## [41] "/title/tt0172495/?ref=adv_li_tt" "/title/tt0120689/?ref=adv_li_tt"
## [43] "/title/tt0120586/?ref=adv_li_tt" "/title/tt0110357/?ref=adv_li_tt"
## [45] "/title/tt0103064/?ref=adv_li_tt" "/title/tt0095765/?ref=adv_li_tt"
## [47] "/title/tt0095327/?ref=adv_li_tt" "/title/tt0088763/?ref=adv_li_tt"
## [49] "/title/tt0082971/?ref=adv_li_tt" "/title/tt0078788/?ref=adv_li_tt"
```

Now we have our first two vectors one that is the 50 titles, and one that is the 50 corresponding URLs. Seems like a good time to create a dataframe to house these items. As we collect our other pieces of data they can be added to the frame. Adding new columns to a dataframe is easy, you put the name of the dataframe followed by the name you want for the column in brackets and quotes.

```
imdb_top_250 = as.data.frame(titles, stringsAsFactors = FALSE)
imdb_top_250["URLs"] <- urls
```

Here is how the first few items of our data.frame look at this point:

```
head(imdb_top_250)
```

	titles	URLs
1	The Shawshank Redemption	/title/tt0111161/?ref=adv_li_tt
2	The Godfather	/title/tt0068646/?ref=adv_li_tt
3	The Dark Knight	/title/tt0468569/?ref=adv_li_tt
4	The Godfather: Part II	/title/tt0071562/?ref=adv_li_tt
5	The Lord of the Rings: The Return of the King	/title/tt0167260/?ref=adv_li_tt
6	Pulp Fiction	/title/tt0110912/?ref=adv_li_tt

MORE DATA!

The other items we want to obtain are: the year of the movie, the rating, runtime, genre, and IMDB star rating.

For this we need to go back to the website and look at the HTML again. This time we'll use the css selector for this item, obtained from the "inspect" option in Google Chrome. We'll start with year. I review the selector path when I inspect the year and determine that the year is within the span called "lister-item-year.text-muted.unbold."

```

    </span>
    <a href="/title/tt0111161/?ref=adv_li_tt">The
    Shawshank Redemption</a>
...
    <span class="list-item-year text-muted unbold">(1994)
    </span> == $0
  </h3>
  <p class="text-muted ">...</p>
  <div class="ratings-bar">...</div>
  <p class="text-muted">...</p>
  <p class="text-muted">...</p>
  <p class="sort-num_votes-visible">...</p>
</div>
</div>
<div class="list-item mode-advanced">...</div>
<div class="list-item mode-advanced">...</div>
<div class="list-item mode-advanced">...</div>
... #main div div div div div h3 span.list-item-year.text-muted.unbold

```

Now we'll use the `html_node` function to pull back the first item to make sure we're pulling the right information. Note that I'm calling my function again on the entire `sitehtml`, not `body_rows`. `body_rows` was a specific subset of the full `sitehtml` that only contained the title information.

Here I am using the `%>%` operator to string together multiple functions. This is called a pipe operator from the package `magrittr` which is built into `rvest` as both are created by the same developer. This operator allows us to perform sequential actions without nesting functions inside each other, which makes our code more readable.

Without the pipe operator, our code to perform the two actions would look like this with the function calls "nested" inside of each other:

```
html_text(html_node(sitehtml, 'span.list-item-year.text-muted.unbold'))
```

This is less readable because the LAST function we're using is appearing FIRST in the code.

I first use `html_node` to obtain the node for this span item, then `html_text` to extract the text from that node.

```
html_node(sitehtml, 'span.list-item-year.text-muted.unbold') %>% html_text()
```

```
## [1] "(1994)"
```

I see that I'm correctly getting the year of the first movie, so I update my call to obtain all of the nodes.

```
html_nodes(sitehtml, 'span.list-item-year.text-muted.unbold') %>% html_text()
```

```
## [1] "(1994)" "(1972)" "(2008)" "(1974)" "(2003)" "(1994)" "(1993)"
## [8] "(1966)" "(1957)" "(2010)" "(1999)" "(2001)" "(1994)" "(1980)"
## [15] "(2018)" "(2002)" "(1999)" "(1990)" "(1975)" "(1954)" "(2018)"
## [22] "(2014)" "(2002)" "(2001)" "(1998)" "(1997)" "(1995)" "(1995)"
## [29] "(1994)" "(1991)" "(1977)" "(1946)" "(2016)" "(2018)" "(2014)"
## [36] "(2011)" "(2006)" "(2006)" "(2002)" "(2000)" "(2000)" "(1999)"
## [43] "(1998)" "(1994)" "(1991)" "(1988)" "(1988)" "(1985)" "(1981)"
## [50] "(1979)"
```

Now we've figure out where our years are, we need to build up a call that will pull them, remove the parens, and save them to our dataframe, utilizing the `magrittr` pipe operator to add each piece sequentially.

To remove the parens we'll need to use string functions. So we'll load the `stringr` package.

```
if (!requireNamespace("stringr", quietly = TRUE)) {
  install.packages("stringr")
}
library("stringr")
```

Here we add the function `str_extract` to the end of our set of actions. `str_extract` takes an argument that is called a Regular Expression (regex) which is like a pattern matcher that tells the computer what piece of a string of text we want to extract (keep). Here we want the four numbers without the parentheses. `\\d` tells the computer we want a number and adding `{4}` says we want 4 numbers in a row.

```
html_nodes(sitehtml, 'span.list-item-year.text-muted.unbold') %>% html_text() %>% str_extract("\\d{4}")
```

```
## [1] "1994" "1972" "2008" "1974" "2003" "1994" "1993" "1966" "1957" "2010"
## [11] "1999" "2001" "1994" "1980" "2018" "2002" "1999" "1990" "1975" "1954"
## [21] "2018" "2014" "2002" "2001" "1998" "1997" "1995" "1995" "1994" "1991"
## [31] "1977" "1946" "2016" "2018" "2014" "2011" "2006" "2006" "2002" "2000"
## [41] "2000" "1999" "1998" "1994" "1991" "1988" "1988" "1985" "1981" "1979"
```

Now that we have the years formatted in the way we want them, we simply save it to our dataframe in a column called "year."

```
imdb_top_250["year"] <- html_nodes(sitehtml, 'span.lister-item-year.text-muted.unbold') %>% html_text() %>% str_extract("\\d{4}")
```

Here is how the first 6 rows look now with the years added:

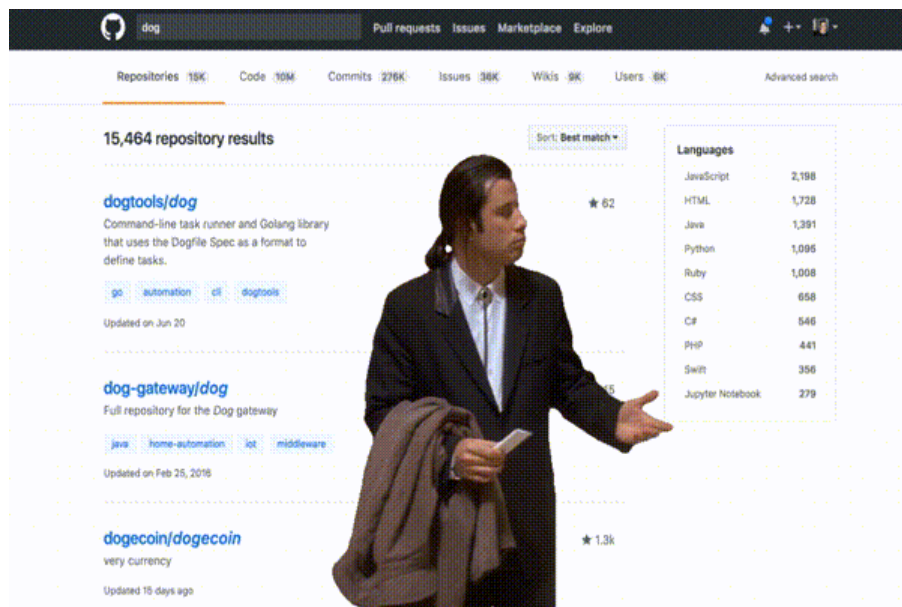
```
head(imdb_top_250)
```

	titles	URLs	year
1	The Shawshank Redemption	/title/tt0111161/?ref_=adv_li_tt	1994
2	The Godfather	/title/tt0068646/?ref_=adv_li_tt	1972
3	The Dark Knight	/title/tt0468569/?ref_=adv_li_tt	2008
4	The Godfather: Part II	/title/tt0071562/?ref_=adv_li_tt	1974
5	The Lord of the Rings: The Return of the King	/title/tt0167260/?ref_=adv_li_tt	2003
6	Pulp Fiction	/title/tt0110912/?ref_=adv_li_tt	1994

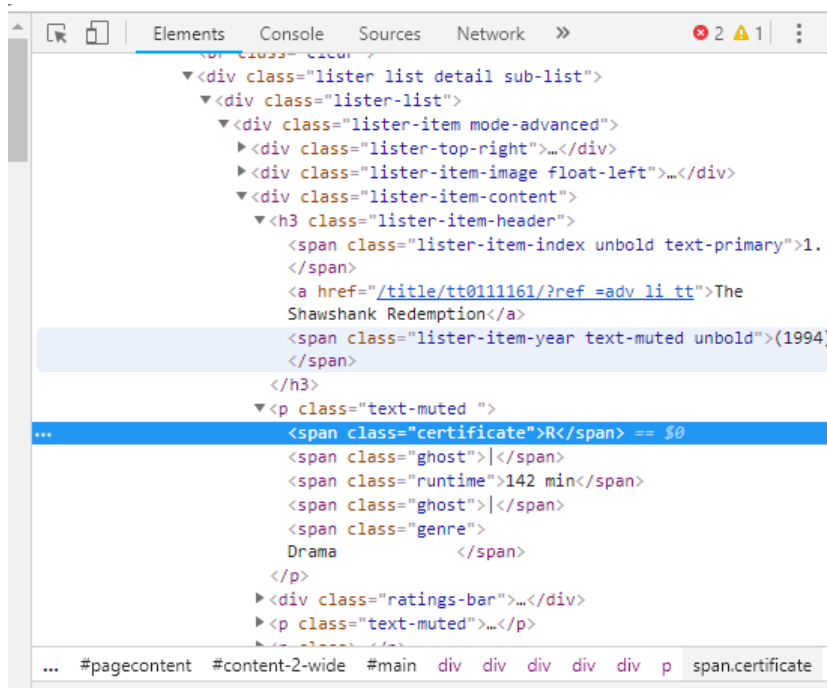
We'll quickly build the rest of our columns...

It's your turn to try. Inspect the website and figure out the selector path for the MPAA rating of the movie (PG, PG-13, R, etc.). Resist the urge to peek below!!!

FIND THE RATING USING INSPECT IN CHROME



AND THE ANSWER IS....



The Rating

We've found that the MPAA rating is contained in a span called certificate. I quickly built the string of `html_nodes` followed by `html_text` to pull out these ratings and add them to our dataframe.

```
imdb_top_250["rating"] <- html_nodes(sitehtml, 'span.certificate') %>% html_text()
```

Revisiting the top 6 rows in the dataframe:

```
head(imdb_top_250)
```

	titles	URLs	year	rating
1	The Shawshank Redemption	/title/tt0111161/?ref=adv_li_tt	1994	R
2	The Godfather	/title/tt0068646/?ref=adv_li_tt	1972	R
3	The Dark Knight	/title/tt0468569/?ref=adv_li_tt	2008	PG-13
4	The Godfather: Part II	/title/tt0071562/?ref=adv_li_tt	1974	R
5	The Lord of the Rings: The Return of the King	/title/tt0167260/?ref=adv_li_tt	2003	PG-13
6	Pulp Fiction	/title/tt0110912/?ref=adv_li_tt	1994	R

Now I'll quickly pull 3 more items into our dataframe:

The runtime

Here for the regex we want just the number `\\d`, of any length `*`, and not the text that says "min." I'll also convert the number as text into a number in case we want to do math with these numbers at some point.

```
imdb_top_250["runtime"] <- html_nodes(sitehtml, 'span.runtime') %>% html_text() %>% str_extract("\\d*") %>% as.numeric()
```

Genre

Some movies have more than one genre, but right now we'll just keep them as one text string. We do have to trim the leading whitespace using `str_trim` also from the `stringr` package to remove any extra whitespace.

```
imdb_top_250["genres"] <- html_nodes(sitehtml, 'span.genre') %>% html_text() %>% str_trim()
```


IMDB star rating

We'll pull out the div with the rating, then the rating itself, which is the text inside the "strong" tag within that div. When we scrape the site, these numbers are stored as text, so we'll use "as.numeric" to covert it to a number.

```
imdb_top_250["stars"] <- html_nodes(sitehtml, 'div.inline-block.ratings-imdb-rating') %>% html_node ("strong") %>% html_text() %>% as.numeric()
```

At this point we have the 7 pieces of information (columns) collected for the 50 movies on the first page.

ALL THE COLUMNS

Let's take a peek at the first few items in our dataframe now:

```
head(imdb_top_250)
```

	titles	URLs	year	rating	runtime	genres	stars
1	The Shawshank Redemption	/title/tt0111161/?ref=adv_li_tt	1994	R	142	Drama	9.3
2	The Godfather	/title/tt0068646/?ref=adv_li_tt	1972	R	175	Crime, Drama	9.2
3	The Dark Knight	/title/tt0468569/?ref=adv_li_tt	2008	PG-13	152	Action, Crime, Drama	9.0
4	The Godfather: Part II	/title/tt0071562/?ref=adv_li_tt	1974	R	202	Crime, Drama	9.0
5	The Lord of the Rings: The Return of the King	/title/tt0167260/?ref=adv_li_tt	2003	PG-13	201	Adventure, Drama, Fantasy	8.9
6	Pulp Fiction	/title/tt0110912/?ref=adv_li_tt	1994	R	154	Crime, Drama	8.9

To collect the information for the other 200 movies, we will have to navigate to the next page and repeat the process. However, this time we already know where our information is, and we already have our dataframe set up, so we'll simply append these results to the bottom.

Navigate to the next page

In order to navigate within the site, we need to first set up a session. Remember we saved the URL of the page of IMDB's top 250 in a variable called "site."

```
sess <- html_session(site)
is.session(sess)
```

```
## [1] TRUE
```

```
sess <- sess %>% follow_link("Next")
```

```
## Navigating to /search/title?groups=top_250&sort=user_rating,desc&start=51&ref=adv_nxt
```

With the session activated, we don't need to use the `read_html` step. Let's read the list number of the first item on the page to ensure we're on items 51 through 100.

```
html_node(sess, "span.lister-item-index.unbold.text-primary") %>% html_text()
```

```
## [1] "51."
```

Now we're quickly going to scrape the 7 columns of data as we did above, saving each as a vector. These are the code pieces we built above. I've added "lister-item-header" as a qualifier to `<h3 a>` in order to pull only the movie titles and not the additional `<h3 a>` items from the bottom of the page (the extra 5 items we had removed above).

```

titles <- html_nodes(sess, "h3.lister-item-header a") %>% html_text()
urls <- html_nodes(sess, "h3.lister-item-header a") %>% html_attr("href")
years <- html_nodes(sess, 'span.lister-item-year.text-muted.unbold') %>% html_text() %>% str_extract("\\d{4}")
ratings <- html_nodes(sess, 'span.certificate') %>% html_text()
runtimes <- html_nodes(sess, 'span.runtime') %>% html_text() %>% str_extract("\\d*") %>% as.integer()
genres <- html_nodes(sess, 'span.genre') %>% html_text() %>% str_trim()
stars <- html_nodes(sess, 'div.inline-block.ratings-imdb-rating') %>% html_node("strong") %>% html_text() %>% as.numeric()

```

Now we need to bind the columns together creating a data.frame called page with just items 51-100.

```
page <- data.frame(titles, urls, years, ratings, runtimes, genres, stars)
```

```
## Error in data.frame(titles, urls, years, ratings, runtimes, genres, stars): arguments imply differing number of rows: 50, 48
```

OOPS - we got an error.

```
length(titles)
```

```
## [1] 50
```

```
length(urls)
```

```
## [1] 50
```

```
length(years)
```

```
## [1] 50
```

```
length(ratings)
```

```
## [1] 48
```

```
length(runtimes)
```

```
## [1] 50
```

```
length(genres)
```

```
## [1] 50
```

```
length(stars)
```

```
## [1] 50
```

It looks like two of the movies on this page have no rating listed, and the object the rating is supposed to be in is not even present. Because the object isn't present, we don't even get a blank, therefore having only 48 ratings with 50 movies. Our method of scraping was naive to this issue. We're going to slightly reformat the scraper to read each movie one by one and pick out the pieces we need.

We will select the larger node that contains the information about each movie, then pick out the attributes that correspond to each of our 7 items from that larger node. When those nodes are not present it will automatically include an NA value in that row.

First, create a completely empty dataframe...

```
page <- data.frame()
```

...and read each of the movies off of the page into a vector where each element is the HTML for the entire movie "block."

IMDb "Top 250" (Sorted by IMDb Rating Descending)

1-50 of 250 titles. | [Next »](#)View Mode: [Compact](#) | [Detailed](#)
 Sort by: [Popularity](#) | [A-Z](#) | [User Rating](#) ▼ | [Number of Votes](#) | [US Box Office](#) | [Runtime](#) | [Year](#)

div.lister-item.mode-advanced

 620 × 203 Your Rating


1. The Shawshank Redemption (1994)

R | 142 min | Drama

★ 9.3

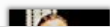
☆ Rate this

80 Metascore

Two imprisoned men bond over a number of years, finding solace and eventual redemption through acts of common decency.

Director: [Frank Darabont](#) | Stars: [Tim Robbins](#), [Morgan Freeman](#), [Bob Gunton](#), [William Sadler](#)

Votes: 2,056,321 | Gross: \$28.34M



2. The Godfather (1972)

```
movies_html <- html_nodes(sess, '.mode-advanced')
```

Check that reading within the first element of the movie vector we can read the node we need using the same node descriptors from above.

```
movies_html[1] %>% html_node("h3.lister-item-header a")
```

```
## {xml_nodeset (1)}
## [1] <a href="/title/tt0078748/?ref=adv_li_tt">Alien</a>
```

So instead of scraping 50 titles, then 50 URLs, etc. from the website, we'll scrape the block containing each movie, then 1 by 1 we'll scrape the title, URL, etc. from each movie block. For this we'll loop through the 50 movie blocks on the page to read each of the 7 nodes and append it to our empty dataframe. This way when an item returns an NA (it's not found in that movie block), it is included in that row as an NA.

```
len <- length(movies_html) # get the length of the movie blocks vector as a variable
for (i in 1:len) { # Loop through each movie block one by one
  #set the 7 elements for this movie
  title <- movies_html[i] %>% html_node("h3.lister-item-header a") %>% html_text()
  url <- movies_html[i] %>% html_node("h3.lister-item-header a") %>% html_attr("href")
  year <- movies_html[i] %>% html_node('span.lister-item-year.text-muted.unbold') %>% html_text() %>% str_extract("\\d{4}")
  rating <- movies_html[i] %>% html_node('span.certificate') %>% html_text()
  runtime <- movies_html[i] %>% html_node('span.runtime') %>% html_text() %>% str_extract("\\d*") %>% as.integer()
  genre <- movies_html[i] %>% html_node('span.genre') %>% html_text() %>% str_trim()
  star_rating <- movies_html[i] %>% html_node('div.inline-block.ratings-imdb-rating') %>% html_node("strong") %>% html_text() %>% as.numeric()

  #create the row for this movie by column binding (cbind) the 7 items
  movie_row <- cbind(title, url, year, rating, runtime, genre, star_rating)
  #append the new row to the page df
  page <- rbind(page, movie_row)
}
```

Let's take a peek at the last few items in our dataframe from this page:

```
tail(page)
```

title	url	year	rating	runtime	genre	star_rating
Children of Heaven	/title/tt0118849/?ref=adv_li_tt	1997	PG	89	Drama, Family	8.3
The Bandit	/title/tt0116231/?ref=adv_li_tt	1996	NA	128	Crime, Drama, Thriller	8.3
Toy Story	/title/tt0114709/?ref=adv_li_tt	1995	G	81	Animation, Adventure, Comedy	8.3
Reservoir Dogs	/title/tt0105236/?ref=adv_li_tt	1992	R	99	Crime, Drama, Thriller	8.3
Full Metal Jacket	/title/tt0093058/?ref=adv_li_tt	1987	R	116	Drama, War	8.3
Amadeus	/title/tt0086879/?ref=adv_li_tt	1984	R	160	Biography, Drama, History	8.3

Now that we have the items from the second page (movies 51-100), we'll append that to our first 50 items to create our dataframe of the first 100 items.

In order to do this, the column names in both dfs need to match. (I could have used the same column names, BTW, but I was being lazy while coding.) It's easy to fix using the `colnames` function.

```
colnames(page) <- colnames(imdb_top_250)
```

```
imdb_top_250 <- rbind(imdb_top_250, page)
dim(imdb_top_250)
```

```
## [1] 100 7
```

Now we can look at the first items and the last items of our `imdb_top_250` dataframe to ensure that it starts with the first items from the first page and ends with the last items from the second page we just scraped:

HEAD:

	titles	URLs	year	rating	runtime	genres	stars
1	The Shawshank Redemption	/title/tt0111161/?ref_=adv_li_tt	1994	R	142	Drama	9.3
2	The Godfather	/title/tt0068646/?ref_=adv_li_tt	1972	R	175	Crime, Drama	9.2
3	The Dark Knight	/title/tt0468569/?ref_=adv_li_tt	2008	PG-13	152	Action, Crime, Drama	9
4	The Godfather: Part II	/title/tt0071562/?ref_=adv_li_tt	1974	R	202	Crime, Drama	9
5	The Lord of the Rings: The Return of the King	/title/tt0167260/?ref_=adv_li_tt	2003	PG-13	201	Adventure, Drama, Fantasy	8.9
6	Pulp Fiction	/title/tt0110912/?ref_=adv_li_tt	1994	R	154	Crime, Drama	8.9

TAIL:

	titles	URLs	year	rating	runtime	genres	stars
95	Children of Heaven	/title/tt0118849/?ref_=adv_li_tt	1997	PG	89	Drama, Family	8.3
96	The Bandit	/title/tt0116231/?ref_=adv_li_tt	1996	NA	128	Crime, Drama, Thriller	8.3
97	Toy Story	/title/tt0114709/?ref_=adv_li_tt	1995	G	81	Animation, Adventure, Comedy	8.3
98	Reservoir Dogs	/title/tt0105236/?ref_=adv_li_tt	1992	R	99	Crime, Drama, Thriller	8.3
99	Full Metal Jacket	/title/tt0093058/?ref_=adv_li_tt	1987	R	116	Drama, War	8.3
100	Amadeus	/title/tt0086879/?ref_=adv_li_tt	1984	R	160	Biography, Drama, History	8.3

Finally, we can wrap that "next page process" into one big loop to get the last 3 pages we need.

```

for(i in 1:3){
  sess <- sess %>% follow_link("Next")
  page <- data.frame()
  movies_html <- html_nodes(sess, '.mode-advanced')
  len <- length(movies_html) # get the length of the movie blocks vector as a variable
  for (i in 1:len) { # Loop through each movie block one by one
    #set the 7 elements for this movie
    title <- movies_html[i] %>% html_node("h3.lister-item-header a") %>% html_text()
    url <- movies_html[i] %>% html_node("h3.lister-item-header a") %>% html_attr("href")
    year <- movies_html[i] %>% html_node('span.lister-item-year.text-muted.unbold') %>% html_text() %>% str_extract("\\d{4}")
  )
  rating <- movies_html[i] %>% html_node('span.certificate') %>% html_text()
  runtime <- movies_html[i] %>% html_node('span.runtime') %>% html_text() %>% str_extract("\\d*") %>% as.integer()
  genre <- movies_html[i] %>% html_node('span.genre') %>% html_text() %>% str_trim()
  star_rating <- movies_html[i] %>% html_node('div.inline-block.ratings-imdb-rating') %>% html_node("strong") %>% html_text() %>% as.numeric()

  #create the row for this movie by column binding (cbind) the 7 items
  movie_row <- cbind(title, url, year, rating, runtime, genre, star_rating)
  #append the new row to the page df
  page <- rbind (page, movie_row)
}
colnames(page) <- colnames(imdb_top_250)
imdb_top_250 <- rbind(imdb_top_250, page)
}
dim(imdb_top_250) # print final dimensions

```

```
## [1] 250 7
```

HEAD:

	titles	URLs	year	rating	runtime	genres	stars
1	The Shawshank Redemption	/title/tt0111161/?ref=adv_li_tt	1994	R	142	Drama	9.3
2	The Godfather	/title/tt0068646/?ref=adv_li_tt	1972	R	175	Crime, Drama	9.2
3	The Dark Knight	/title/tt0468569/?ref=adv_li_tt	2008	PG-13	152	Action, Crime, Drama	9
4	The Godfather: Part II	/title/tt0071562/?ref=adv_li_tt	1974	R	202	Crime, Drama	9
5	The Lord of the Rings: The Return of the King	/title/tt0167260/?ref=adv_li_tt	2003	PG-13	201	Adventure, Drama, Fantasy	8.9
6	Pulp Fiction	/title/tt0110912/?ref=adv_li_tt	1994	R	154	Crime, Drama	8.9

TAIL:

	titles	URLs	year	rating	runtime	genres	stars
245	Groundhog Day	/title/tt0107048/?ref=adv_li_tt	1993	PG	101	Comedy, Fantasy, Romance	8
246	Beauty and the Beast	/title/tt0101414/?ref=adv_li_tt	1991	G	84	Animation, Family, Fantasy	8
247	The Terminator	/title/tt0088247/?ref=adv_li_tt	1984	R	107	Action, Sci-Fi	8
248	Jaws	/title/tt0073195/?ref=adv_li_tt	1975	PG	124	Adventure, Drama, Thriller	8
249	The Exorcist	/title/tt0070047/?ref=adv_li_tt	1973	R	122	Horror	8
250	The Wizard of Oz	/title/tt0032138/?ref=adv_li_tt	1939	PG	102	Adventure, Family, Fantasy	8

Now that we have our data we can do things like print just the rows that are classified as Comedies.

```
comedies <- imdb_top_250[grep("Comedy", imdb_top_250$genre), ]
```

```
head(comedies)
```

	titles	URLs	year	rating	runtime	genres	stars
26	Life Is Beautiful	/title/tt0118799/?ref=adv_li_tt	1997	PG-13	116	Comedy, Drama, Romance	8.6
36	The Intouchables	/title/tt1675434/?ref=adv_li_tt	2011	R	112	Biography, Comedy, Drama	8.5
48	Back to the Future	/title/tt0088763/?ref=adv_li_tt	1985	PG	116	Adventure, Comedy, Sci-Fi	8.5
56	The Great Dictator	/title/tt0032553/?ref=adv_li_tt	1940	Passed	125	Comedy, Drama, War	8.5
57	Modern Times	/title/tt0027977/?ref=adv_li_tt	1936	G	87	Comedy, Drama, Family	8.5
58	City Lights	/title/tt0021749/?ref=adv_li_tt	1931	G	87	Comedy, Drama, Romance	8.5

Or we can find the distribution of runtimes:

```
summary(as.numeric(imdb_top_250$runtime))
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      45.0   107.0   125.5   129.4   145.8   321.0
```

```
hist(as.numeric(imdb_top_250$runtime), main="Runtimes of IMDB Top Rated Movies", xlab="Runtime (in minutes)", col="darkmagenta", breaks=40, xlim=c(40,330))
```

Runtimes of IMDB Top Rated Movies

